```
//*****************************************************************************************
//*****************************************************************************************
//                                                                                       **
//                        PROCESOR LPC2129/01                                            **
//      funkcjonalnosci:                                                                 **
//          - Znajdowanie tokenów w łańcuchu znakowym                                    **
//          - Zamiana łańcucha znakowego na kod słowa kluczowego                         **
//          - Dekodowanie tokenu                                                         **
//          - Dekodowanie komunikatu                                                     **
//                                                                                       **
//*****************************************************************************************


#define NULL '\0'
#define MAX_TOKEN_NR 3
#define MAX_KEYWORD_STRING_LTH 10
#define MAX_KEYWORD_NR 3

enum TokenType { KEYWORD, NUMBER, STRING };
enum KeywordCode { LD, ST, RST };

union TokenValue {
    enum KeywordCode eKeyword;
    unsigned int uiNumber;
    char *pcString;
};

struct Token {
    enum TokenType eType;
    union TokenValue uValue;
};

struct Keyword {
    enum KeywordCode eCode;
    char cString[MAX_KEYWORD_STRING_LTH + 1];
};

struct Keyword asKeywordList[MAX_KEYWORD_NR] = {
{RST, "reset"},
{LD, "load" },
{ST, "store"}
};
```

```
struct Token asToken[MAX_TOKEN_NR];
unsigned char ucTokenNr;

enum State { TOKEN, DELIMITER };
unsigned char ucFindTokensInString (char *pcString) {

    enum State eState = DELIMITER;
    unsigned char ucCharValue;
    unsigned char ucCharCounter;

    ucTokenNr = 0;

    for(ucCharCounter = 0;; ucCharCounter++) {
        ucCharValue = pcString[ucCharCounter];

        switch (eState) {
            case DELIMITER:
                if(NULL == ucCharValue) {
                    return ucTokenNr;
                } else if(' ' == ucCharValue) {}
                else if(MAX_TOKEN_NR == ucTokenNr) {
                    return ucTokenNr;
                } else {
                    eState = TOKEN;
                    asToken[ucTokenNr].uValue.pcString = pcString + ucCharCounter;
                    ucTokenNr++;
                }

                break;
            case TOKEN:
                if(NULL == ucCharValue) {
                    return ucTokenNr;
                } else if(' ' == ucCharValue) {
                    eState = DELIMITER;
                } else {};

                break;
        }
    }
}
```

```
enum Result { OK, ERROR };
enum Result eSringToKeyword (char pcStr[], enum KeywordCode *peKeywordCode) {

    unsigned char ucKeywordCounter;

    for(ucKeywordCounter = 0; ucKeywordCounter < MAX_KEYWORD_NR; ucKeywordCounter++) {
        if(EQUAL == eCompareString(pcStr, asKeywordList[ucKeywordCounter].cString)) {
            *peKeywordCode = asKeywordList[ucKeywordCounter].eCode;
            return OK;
        }
    }

    return ERROR;
}



void DecodeTokens(void) {

    unsigned char ucTokenCounter;
    struct Token *psToken;

    for(ucTokenCounter = 0; ucTokenCounter < ucTokenNr; ucTokenCounter++) {
        psToken = &asToken[ucTokenCounter];

        if(OK == eSringToKeyword(psToken->uValue.pcString, &psToken->uValue.eKeyword)) {
            psToken->eType = KEYWORD;
        } else if (OK == eHexStringToUInt(psToken->uValue.pcString, &psToken->uValue.uiNumber)) {
            psToken->eType = NUMBER;
        } else {
            psToken->eType = STRING;
        }
    }
}



void DecodeMsg(char *pcString) {

    ucTokenNr = ucFindTokensInString(pcString);
    ReplaceCharactersInString(pcString, ' ', NULL);
    DecodeTokens();
}
```